



شماره دانشجویی:

۸۱۰۱۰۱۴۹۲



تمرین شماره ۱

تئوری زنجیره‌های بلوکی
نام و نام خانوادگی: علیرضا کریمی

سوال ۱

(a)

خب برای حل سوال کافیست مثالی از یک تابع هش بزنی که *collision-resistant* باشد اما امنیت خوبی برای *proof-of-work* نداشته باشد. برای این کار تابع هش خود را به این شکل تعریف می‌کنیم:

$$H'(m) = (R(m), H(m))$$

که فرض می‌کنیم $H(m)$ همان تابع هش امن اصلی ما با خروجی n بیت است. به طور مثال همان $SHA-256$ هست با ۲۵۶ بیت خروجی. تابع هش $R(m)$ نیز یک تابع با خروجی هش ۳۲ بیتی است. پس در نتیجه، تابع هش جدید ما که از *concat* بین این دو ساخته شده، $256 + 32 = 288$ بیت خروجی دارد.

از آنجایی که $H(m)$ به میزان خیلی خوبی *collision-resistant* است، بنابراین تابع $H'(m)$ ساخته ما نیز همین‌گونه است و به سختی می‌توان دو ورودی m_1 و m_2 پیدا کرد که هش یکسان داشته باشند، چون این کار برای تابع هش اصلی سخت بود و این هش جدید ما یک *superclass* از آن است. اما این تابع هش جدید ساخته شده، نسبت به *PoW* امنیت مناسبی ندارد.

فرض کنید که ما می‌خواهیم که مثلاً ۳۲ بیت بزرگ عدد هش خروجی، صفر باشد. در هش اصلی که داشتیم، ماینر باید تمام حالات ممکن 2^{256} را چک کند، و دامنه چک کردن ما به همان اندازه است. اما وقتی $H'(m)$ را ساختیم، آن ماینر تنها کافیست که ۳۲ بیت بالاتر که مربوط به $R(m)$ می‌شود را در دامنه خود برای چک کردن حالات مختلف قرار دهد. بنابراین، به مراتب محاسبات کمتری (به واسطه دامنه کمتر) انجام می‌دهد تا هدف یکسان *PoW* را برآورده کند. درواقع فضای حالتی که باید ماینر چک کند، از 2^{256} به میزان 2^{32} می‌رسد. به طور مثال، اگر برای رسیدن به مقدار مناسب در هش اولیه به 2^{56} محاسبه نیاز بود، در هش جدید، ماینر می‌تواند مقدار $R(m)$ را تنها در 2^{32} بررسی کند، و در نتیجه در مورد تابع هش اولیه، سختی مسئله از 2^{56} به 2^{24} کاهش می‌یابد که امنیت *PoW* را تضعیف می‌کند. یعنی کل محاسبات ما به نوعی از 2^{56} به $2^{32} + 2^{24}$ تبدیل شده که خیلی کمتر است.

(b)

اتریوم: برای اتریوم از الگوریتم *ethash* استفاده می‌شود که الگوریتمی *memory - intensive* هست که به نوعی *asic - resistance* محسوب می‌شود یعنی به سختی می‌توان *asic* بهینه شده مربوط به آن را ساخت چون باید در کنار *asic* حافظه زیادی نیز قرار بدهیم که برای ماینرها به صرفه نیست.

لایت کوین: لایت کوین از الگوریتم *scrypt* استفاده می‌کند که در ابتدا به منظور *asic - resistance* بودن طراحی شد اما به دلیل باگی در کد (که با جای استفاده از یک مقدار متغیر، مقدار ثابت ۱۰۲۴ را استفاده کرده بود و برای همین آن چنان هم *memory - hardness* بالایی پیدا نمی‌کند، برای همین توانسته‌اند برایش *asic* طراحی کنند. زی‌کش: برای آن از الگوریتم *equihash* استفاده می‌شود که باز هم یک الگوریتم *memory - hard* و *asic - resistance* است.

مونرو: از الگوریتم *randomx* استفاده می‌کند که *asic - resistance* است.

نکته‌ای که هست در مورد همه این موارد این است که بسیاری از الگوریتم‌ها در تلاش هستند تا *asic resistance* باشند. چون اگر که قرار باشد که *asic*ها فقط ماین کنند، به نوعی تمرکز ایجاد می‌شود و اون هدف غیرمتمرکز بودن زیر سوال می‌رود. با کمک این الگوریتم‌ها، هر شخص عادی هم می‌تواند شانس خوبی در مشارکت در انجام آن داشته باشد.

از طرفی بعضی از کوین‌ها و الگوریتم‌های آنها برای کاربرد های *dapp* ساخته شدند و *scalability* و *performance* در بعضی الگوریتم‌ها مهم تر است. برعکس *bitcoin* که هر ده دقیقه یک بلاک ماین می‌کند و خب سرعت پایینی محسوب می‌شود. و درواقع می‌توان دید که بعضی از الگوریتم‌ها چون برای منظور خاصی بهینه شده‌اند، سرعت زیادی دارند یا انرژی و توان کمتری مصرف می‌کنند.

از طرفی، با اینکه الگوریتم‌های مورد استفاده در بیت کوین امنیت خوبی دارند، اما برای بعضی از کاربردها به امنیت بیشتری نیاز بود و برای همین الگوریتم‌هایی با تمرکز بر امنیت بیشتر ساخته شد.

سوال ۲

(a)

برای حالتی که درخت دودویی داشتیم، برای هر لول به یک هش برادرمان نیاز داریم که در مجموع به تعداد کل لول‌ها به هش نیاز داریم. تعداد لول‌ها هم برابر $\log_2 n$ بود. حال که درخت ما k تایی است، برای رفتن به لول بالاتر، باید هش تمام برادرانمان در آن لول را داشته باشیم که می‌شود $k - 1$ تا هش در آن لول. بعد هش خودمان را حساب می‌کنیم با آن $k - 1$ تا هش برادرانمان، *concat* می‌کنیم. برای مرحله بعد نیاز داریم که هش این را حساب کرده و با $k - 1$ تا هش در لول بالاتر این *concat* را انجام بدهیم. این کار را باید آنقدر انجام دهیم تا به ریشه برسیم و بعد هش

آن را گزارش می‌کنیم. از طرفی تعداد کل لول‌ها هم برابر می‌شود با $\log_k n$. پس در کل به تعداد $\log_k n \times (k - 1)$ تا هش نیاز داریم برای محاسبه هش ریشه.

البته اگر بخواهیم هش‌هایی را هم که خودمان حساب کنیم، باید با تعداد لول‌ها هش محاسبه کنیم. چون هر لول که بالا می‌رویم یک هش می‌گیریم. پس علاوه بر $\log_k n \times (k - 1)$ تعداد هش که می‌گیریم، به اندازه $\log_k n$ تا هش هم خودمان محاسبه می‌کنیم که در مجموع تعداد کل هش‌ها به طور دقیق‌تر برابرست با: $k \times \log_k n$ این فرمول کلی خواهد بود. (این مقدار برای k های بزرگ، به n میل می‌کند).

(b)

اگر روی نمودار نیز مشاهده کنید، خواهید دید که در اغلب موارد، مقدار $2 \times \log_2 n$ از تمام مقادیر $k \times \log_k n$ کمتر خواهد بود که این نشان‌دهنده این است که تعداد هش‌های لازم کمتر خواهد بود. اما دقت کنید، با اینکه تعداد کل هش‌ها کمتر است، اما تعداد هش‌هایی که ما باید محاسبه کنیم، بستگی به تعداد لول‌های درخت دارد و هر قدر لول درخت بیشتر، هش‌هایی که از بیرون می‌گیریم، کمتر و هش‌هایی که محاسبه می‌کنیم بیشتر است. هر چقدر k را زیاد کنیم، تعداد هش‌های بیشتری از بیرون می‌گیریم اما چون لول (ارتفاع) درخت کمتر است، هش‌های محاسباتی ما نیز کمتر خواهد شد. منظوری این است که در حالت $k = 2$ ما به تعداد $\log_2 n$ تا هش از بیرون و $\log_2 n$ تا هم هش خودمان محاسبه می‌کردیم تا به ریشه برسیم. اما برای بقیه مقادیر، به تعداد $(k - 1) \times \log_k n$ تا از بیرون می‌گیریم، و خودمان تنها $\log_k n$ تا هش محاسبه می‌کنیم که برای $k > 2$ ، به مراتب از $\log_2 n$ کمتر است.

سوال ۳

(a)

خب طبق مراحل که برای محاسبه آدرس از روی *publickey* داریم [۱]، ابتدا $SHA - 256$ گرفته و سپس $RIPEMD - 160$ می‌گیریم که خروجی آن در نهایت ۱۶۰ بیت خواهد بود. بعد از آن با اینکه یک سری بایت اضافه می‌کنیم که مخصوص بیت‌کوین است و همیشه برای همه آدرس‌ها یکسان است و باز دو بار $SHA - 256$ می‌گیریم اما در هیچ کدام از مراحل بعدی، دامنه فضای حالات ما کوچک‌تر نمی‌شود. پس فضای حالات ما همان ۱۶۰ بیت خروجی $RIPEMD - 160$ خواهد بود. پس اگر یک کاربر از این طریق به یک عدد ۱۶۰ بیتی برسد، احتمال اینکه نفر بعدی نیز به همین عدد برسد، $\frac{1}{2^{160}}$ خواهد بود چون 2^{160} حالت همگن داریم و کاربر اول به احتمال یکسان در هر کدام آنها می‌تواند باشد.

(b)

طبق قضیه *birthday* در صورتی که کل حالات ما 2^{160} باشد (چون ۱۶۰ بیت داریم)، در نتیجه، با انتخاب $2^{160/2} = 2^{80}$ تا ورودی مختلف (در اینجا منظور *publickey* مختلف برای ساخت آدرس است) می‌توانیم به احتمال ۵۰ درصد مطمئن باشیم که دو ورودی هستند که برای آنها تصادم *collision* رخ داده است. دو ورودی مختلف که به یک خروجی یکسان رسیدند. توجه کنید که اگر خروجی $RIPEMD - 160$ یکسان باشد، در نهایت هم آدرس‌های یکسانی ساخته خواهد شد چون روند ایجاد آنها یکی است و از روی همین ۱۶۰ بیت است و چیز خارجی. از طرفی ممکن است خروجی $SHA - 256$ ابتدایی متفاوت باشد اما خروجی $RIPEMD - 160$ آنها یکسان باشد چون فضای حالات $RIPEMD - 160$ از $SHA - 256$ به مراتب کمتر است.

سوال ۴

(a)

خب باید نقاط را در معادله $y^2 = x^3 + x \pmod{23}$ قرار دهیم و ببینیم آیا در معادله صدق می‌کنند یا نه برای $(x, y) = (15, 2)$ داریم:

$$4 \stackrel{?}{=} 15^3 + 15 \pmod{23} \Rightarrow 4 \stackrel{?}{=} 225 \times 15 + 15 \pmod{23} \Rightarrow 4 \stackrel{?}{=} (-5) \times 15 + 15 \pmod{23}$$

$$\Rightarrow 4 \stackrel{?}{=} (-60) \pmod{23} = 9 \pmod{23} \Rightarrow 4 \neq 9 \pmod{23}$$

پس این نقطه روی منحنی نیست.

برای $(x, y) = (1, 18)$ داریم:

$$18^2 = (-5)^2 \stackrel{?}{=} 1^3 + 1 \pmod{23} \Rightarrow 25 \stackrel{?}{=} 2 \pmod{23} \Rightarrow 2 = 2 \pmod{23}$$

پس این نقطه روی منحنی است.

برای $(x, y) = (18, 13)$ داریم:

$$13^2 \stackrel{?}{=} (-5)^3 + (-5) \pmod{23} \Rightarrow 169 \stackrel{?}{=} (-5) \times (-2) + (-5) \pmod{23} \Rightarrow 8 \neq 10 - 5 \pmod{23}$$

پس این نقطه روی منحنی نیست.

(b)

برای نقاط $P_1 = (9, 18)$ و $P_2 = (13, 18)$ داریم:

$$\Delta = \frac{y_2 - y_1}{x_2 - x_1} \pmod{23}$$

$$\Delta = \frac{18 - 18}{13 - 9} = \frac{0}{4} = 0$$

مختصات نقاط جدید به این شکل محاسبه می‌شود:

$$x_3 = \Delta^2 - x_1 - x_2 \pmod{23}$$

$$y_3 = \Delta(x_1 - x_3) - y_1 \pmod{23}$$

پس داریم:

$$x_3 = 0^2 - 9 - 13 \pmod{23} = -22 \pmod{23} = 1 \pmod{23}$$

$$y_3 = 0(9 - x_3) - 18 = -18 \pmod{23} = 5 \pmod{23}$$

و در نهایت نقطه P_3 برابر خواهد بود با:

$$P_3 = (1, 5) \pmod{23}.$$

برای محاسبه P_4 نیز داریم:

$$P_4 = 2 \times P_2 = P_2 + P_2$$

اما محاسبه Δ در این حالت تا حدی متفاوت است چون صورت و مخرج فرمول قبل Δ ، صفر می‌شود، بنابراین برای دو برابر کردن یک نقطه باید از فرمول ذیل استفاده کرد؛ درواقع برای انجام محاسبات دو برابر کردن یک نقطه در معادله

$$y_2 = x_3 + ax + b \pmod{p} \quad P_4 = (x_4, y_4) \text{ خواهیم داشت:}$$

$$\Delta = \frac{2y_2}{3x_2^2 + a} \pmod{p}$$

$$x_4 = \Delta^2 - 2x_2 \pmod{p}$$

$$y_4 = \Delta(x_2 - x_4) - y_2 \pmod{p}$$

پس داریم:

$$\Delta = \frac{2 \times 18}{3 \times 13^2 + 1} = 36 \times 508^{-1} \pmod{23}$$

$$\Rightarrow \Delta = 13 \times 2^{-1} \pmod{23} = 13 \times 12 \pmod{23} = 156 \pmod{23} = (-5) \pmod{23}$$

حال مختصات آن را محاسبه می‌کنیم:

$$x_4 = -5^2 - 2 \times 13 \pmod{23} = -1 \pmod{23} = 22 \pmod{23}$$

$$y_4 = -5(13 - 22) - 18 \pmod{23} = 27 \pmod{23} = 4 \pmod{23}$$

در نتیجه مختصات نقطه P_4 برابر خواهد بود با:

$$P_4 = (22, 4) \pmod{23}$$

سوال ۵

طبق الگوریتم اقلیدسی تعمیم یافته [۲]، می‌توان x و y را به گونه‌ای پیدا کرد که رابطه زیر برقرار باشد

$$ax + by = \gcd(a, b)$$

و در صورتی هم که $\gcd(a, b) = 1$ باشد، خواهیم داشت: [۳]

$$x = a^{-1} \pmod{b}$$

پس ابتدا در هر کدام از موارد باید ب.م.م آن دو عدد گفته شده را بدست بیاوریم و در صورتی که نسبت به هم اول بودند، می‌توان معکوس آن عدد را در مود خواسته شده حساب کرد.

(a

برای پیدا کردن $\gcd(13, 7)$ از الگوریتم اقلیدسی استفاده می‌کنیم

$$13 = 1 \times 7 + 6$$

$$7 = 1 \times 6 + 1$$

$$6 = 6 \times 1 + 0$$

آخرین باقی‌مانده ناتهی، ب.م.م ما خواهد بود، پس ب.م.م آنها برابر است با ۱. حال چون شرایط پیدا کردن عدد معکوس در مود خواسته شده برقرار هست، از الگوریتم اقلیدسی تعمیم یافته برای پیدا کردن معکوس ۷ در مبنای ۱۳ استفاده می‌کنیم. این کار را به کمک معادله های قبلی که نوشتیم بدست می‌آوریم. داریم:

$$1 = 7 - 1 \times 6 = 7 - 1 \times (13 - 1 \times 7)$$

$$\Rightarrow 1 = 2 \times 7 - 1 \times 13$$

$$\Rightarrow 7^{-1} = 2 \pmod{13}$$

(b)

ابتدا باید ب.م.م ۱۲ و ۳ را حساب کنیم. این مقدار برابر خواهد بود با ۳. پس نمی‌توان برای ۳ عدد معکوسی در مود ۱۲ پیدا کرد چرا که هر مضربی از ۳ در مود ۱۲، حاصلش حتما مضربی از ۳ است و به‌هیچ وجه ۱ نخواهد شد.

سوال ۶

طبق [۴]، برای این منظور از این فرآیند استفاده می‌کنیم:

۱. طرفین بر روی پارامترهای الگوریتم p (یک عدد اول بسیار بزرگ) و g (یک *generator*) توافق می‌کنند.
۲. طرفین کلیدهای خصوصی خود را تولید می‌کنند که به ترتیب a ، b و c نامیده می‌شوند. درواقع آلیس کلید خصوصی a و باب کلید خصوصی b و کارول کلید خصوصی c را دارند
۳. آلیس مقدار $g^a \pmod{p}$ را محاسبه کرده و آن را برای باب ارسال می‌کند.
۴. باب مقدار $g^b \pmod{p} = (g^a)^b \pmod{p}$ را محاسبه کرده و آن را برای کارول ارسال می‌کند.
۵. کارول مقدار $g^{abc} \pmod{p} = (g^a)^c \pmod{p}$ را محاسبه کرده و به عنوان کلید مخفی خود استفاده می‌کند.
۶. باب مقدار $g^b \pmod{p}$ را محاسبه کرده و آن را برای کارول ارسال می‌کند.
۷. کارول مقدار $g^{bc} \pmod{p} = (g^b)^c \pmod{p}$ را محاسبه کرده و آن را برای آلیس ارسال می‌کند.
۸. آلیس مقدار $g^{abc} \pmod{p} = g^{bca} \pmod{p} = (g^{bc})^a \pmod{p}$ را محاسبه کرده و به عنوان کلید مخفی خود استفاده می‌کند.

۹. کارول مقدار $g^c \bmod p$ را محاسبه کرده و آن را برای آلیس ارسال می‌کند.

۱۰. آلیس مقدار $(g^c)^a \bmod p = g^{ca} \bmod p$ را محاسبه کرده و آن را برای باب ارسال می‌کند.

۱۱. باب مقدار $(g^{ca})^b \bmod p = g^{cab} \bmod p = g^{abc} \bmod p$ را محاسبه کرده و به عنوان کلید مخفی خود استفاده می‌کند.

یک شنودگر مقادیر $g^a \bmod p, g^b \bmod p, g^c \bmod p, g^{ab} \bmod p, g^{ac} \bmod p$ و $g^{bc} \bmod p$ را مشاهده کرده است، اما نمی‌تواند هیچ ترکیبی از این مقادیر را برای محاسبه‌ی $g^{abc} \bmod p$ به طور کارآمد به دست آورد.

سوال ۷

همانطور که در سوال ۵ حل کردیم، طبق الگوریتم اقلیدسی تعمیم یافته، رابطه به این فرم وجود دارد:

$$ax + by = \gcd(a, b)$$

و در صورتی هم که $\gcd(a, b) = 1$ باشد، خواهیم داشت: [۳]

$$x = a^{-1} \pmod{b}$$

پس با توجه با اینکه $\gcd(e_1, e_2) = 1$ است، بنابراین می‌توان x و y را پیدا کرد به گونه ای که رابطه $e_1x + e_2y = 1$ برقرار باشد که با توجه به آن داریم:

$$x = e_1^{-1} \pmod{e_2} \text{ and } x = e_2^{-1} \pmod{e_1}$$

که محاسبه آن‌ها به کمک همین محاسبات قضیه الگوریتم اقلیدسی تعمیم یافته، قابل انجام است. خب ما به کلیدهای عمومی و پیام‌های رمزگذاری شده توسط آن‌ها دسترسی داریم. فرض کنید داریم:

$$c_1 = m^{e_1} \pmod{n}$$

$$c_2 = m^{e_2} \pmod{n}$$

بنابراین ما فقط به e_1, e_2, c_1 و c_2 دسترسی داریم و باید به کمک آن‌ها به m برسیم. همینطور x و y را هم حساب کردیم. پس می‌توانیم به این شکل به پیام اولیه برسیم:

$$e_1x + e_2y = 1 \Rightarrow m^{e_1x + e_2y} = m^1$$

$$m^{e_1x + e_2y} = m^{e_1x} \times m^{e_2y} = (m^{e_1})^x \times (m^{e_2})^y = c_1^x \times c_2^y \pmod{n}$$

پس کافیست که عبارت $c_1^x \times c_2^y \pmod{n}$ را حساب کنیم تا به پیام m برسیم.

سوال ۸

همانطور که در سوال های ۵ و ۷ توضیح دادیم و همانطور که در RSA طبق قضیه اوایلر داریم، می‌دانیم که:

$$e^{-1} = d \pmod{\phi(n)}$$

یعنی چون پیمانه $\phi(n)$ را داریم، این موضوع می‌تواند به عنوان یک *backdoor* عمل کرده و به کمک کلید عمومی، کلید خصوصی را بدست آوریم. [۵] کلید خصوصی درواقع معکوس کلید عمومی به پیمانه $\phi(n)$ است. می‌توان به کمک الگوریتم اقلیدسی تعمیم‌یافته، کلید خصوصی را از روی کلید عمومی به کمک داشتن مود، حساب کرد. پس در نهایت کافیست که پیام رمز گذاری را که برابرست با M^e ، به توان d برسانیم تا درنهایت به پیام‌مان برسیم چون می‌دانیم که:

$$(M^e)^d = M^{ed} = M \pmod{n}$$

مراجع

- [1] “The math behind the bitcoin blockchain,” <https://www.elliptic.co/blockchain-basics/the-math-behind-the-bitcoin-blockchain>, 2022.
- [2] “Extended euclidean algorithm,” <https://brilliant.org/wiki/extended-euclidean-algorithm>, 2025.
- [3] “Multiplicative inverse under modulo,” <https://www.geeksforgeeks.org/multiplicative-inverse-under-modulo>, 2024.
- [4] “Diffie-hellman key exchange,” <https://en.wikipedia.org/wiki/Diffie-Hellman-key-exchange>, 2025.
- [5] “RSA explanation,” <https://youtu.be/tCs-lm4Pi58?si=fvZCveQS-Rtjuo>, 2023.